

Natural Language Processing

Lecture notes from my graduate level Quantum Computing class taught by professor James H. Martin at the University of Colorado Boulder. This course covers a range of topics in natural language processing including, but not limited to: probabilistic language modeling, text classification, vector semantics, neural networks, large language models, and transformer architecture. The textbook used is the 3rd Edition of *Speech and Language Processing* by Dan Jurafsky and James H. Martin. The draft of the 3rd edition can be found here: <https://web.stanford.edu/~jurafsky/slp3/>

Disclaimer: This document are notes taken by the author (me) a student in graduate school as way to internalize the material I am learning in this class. As such, these notes will inevitably contain mistakes such as errors, typos, etc. as well as personal anecdotes, quips, opinions, and biases. Do what you will with that information, and as a professor once said to me "It's better not to think about it too much."

Contents

1	Large Language Models	1
1.1	What is NLP?	1
1.2	The Three Pillars of Modern NLP	1
1.3	The entire brief history of speech and language processing (I guess)	1
1.3.1	Foundations of NLP	1
1.3.2	The Symbolic Turn	2
1.3.3	Oh my gosh! Empiricism is back!	2
1.3.4	Just Chat it.	2
1.4	Linguistic Structure	2
1.4.1	Syntactic structure	2
1.4.2	Coreference	2
2	Words and Tokens	3
2.1	Morphology	3
2.1.1	Language classification	4
2.1.2	Morphological Typology	4
2.2	So, what is a word?	4
2.2.1	Vocabulary	4
2.2.2	Corpora	4
2.3	Heap's Law	5
2.4	Text Normalization and Tokenization	5
2.4.1	Issues in Normalization	5
2.4.2	Sentence Segmentation	6
2.4.3	Multilingual issues in Normalization	6
2.4.4	Tokenization	6
2.5	Byte-Pair Encoding (BPE)	6
2.5.1	Multilingual issues with Tokenization	7
2.5.2	Does vocab size matter?	8
3	N-Gram Language Models	8
3.1	What is a language model?	8
3.1.1	Word Prediction	8
3.1.2	The Markov assumption	8
3.2	Smoothing	9
3.2.1	Additive smoothing	9
3.3	Evaluating Language Models	10
3.4	Log probabilities	10
3.4.1	Perplexity and entropy	10
4	Logistic Regression and Text Classification	11

1 Large Language Models

A large language model is a computational system that can predict the next word from previous words. Given a context of previous words a language model assigns probabilities what the next word will be.

For example, when given the sentence "so long and thanks for...", the next word that comes after that is most likely to be guess "all" or "the" (my mind immediately went to Fall Out Boy's song *Thnks fr th Mmrs*, "thanks for the memories"). Unlikely words would probably be "diagonalize" or "bipartite".

The language model is a large neural network that takes in context and returns a probability of the next word. We can also use this prediction to **generate** text by sampling from a distribution (we just generate a random number and choose a word based on the probability assigned to it).

1.1 What is NLP?

Any language tech involved with generating, manipulating, etc. of text.

- e.g. spellcheck, dictation, translation, text-to-speech

Computational Linguistics: Modeling human language using computers so we can understand how languages work

- linked to psychology, philosophy, and cognitive science.

1.2 The Three Pillars of Modern NLP

1. **Probabilistic Language Models:** systems that assign probability distributions over possible next words.
2. **Neural Networks:** the fundamental computational mechanism. Modeled after human brain neurons turning into deep architectures optimized via gradient descent.
3. **Embeddings:** vector-based representations where meaning is mapped as a point in high-dimensional continuous space.

1.3 The entire brief history of speech and language processing (I guess)

The central ideas of NLP lie way back in the 1940s to the early 60s.

1.3.1 Foundations of NLP

- Shannon (1951) developed ideas about word prediction
- Labs at Cornell and Stanford were applying small NNs to computational tasks.
- In 1957 psychologist Osgood proposed that a meaning of a word could be modeled by a point in space
 - Linguists proposed that a word's meaning is related to its context.

1.3.2 The Symbolic Turn

- Modeling language in terms of structure appeared.
- Chomsky argued against statistical modeling and neural models of a language which led to a shift in computer science, AI, and linguistic and cognitive sciences.
- Symbolic approaches become more popular.

1.3.3 Oh my gosh! Empiricism is back!

The field moved back to statistical and neural models (thanks, Chomsky) lasting from 1975 and 1988 to 2017.

- Speech recognition from IBM became a thing.
 - N-gram models and algorithms were developed for this purpose
- Improved algorithms for NN training based on error backpropagation became popular.
 - Led to connectionist/parallel distributed processing
- First neural language models was proposed in 2003.
- Transformer architecture becomes popular for general-purpose neural architecture for various tasks

1.3.4 Just Chat it.

Before prompting, you had to label training data to train a classifier to assign a label to data. Now you can just prompt! What is prompting? Given an algorithm that can predict words, just type in a question and append something like "the answer is" and it would predict the answer!

After this, NLP became about generation rather than algorithms that can parse/interpret text. This paved the way for **generative AI** which focuses on generation of text, images, video, speech, etc.

1.4 Linguistic Structure

Understanding human language and their properties are important for studying speech and language processing.

1.4.1 Syntactic structure

Abstract grammar relationships between words or phrases. For example, verbs have subjects and direct objects

She announced this in January

Detecting these relationships is what **parsing** can do.

1.4.2 Coreference

When a text talks about a person, it can refer to them in many ways.

Victoria, the CFO of J.P Morgan, saw her pay jump from \$2.4 million, as the 32-year old became the company's president

"Victoria" is the "CFO" and the "32-year old". They refer to the same entity "Victoria", just with different ways of saying it.

2 Words and Tokens

- drives LLMS these days
- fundamental bridge from linguistic forms to underlying conceptual representations
- **type**: entries in vocab/lexicon
- how do we manage words/tokens?

2.1 Morphology

- the study of the ways words are built up from smaller units called morphemes.
- **Two classes of morphemes**:
 - **stems**: the core meaning-bearing unit
 - **affixes**: bits and pieces that attach to stems but do not change the core meaning of the word example: "star_s" with the core being "star" and the "-s" being the plural affix we attach.

Inflectional morphology

The combination of stems and affixes where the resulting word...

- has the same word class
- serves a grammatical purpose that is
 - different from the original
 - transparently related to the original
 - * "walk" + "ing" = walking
 - * "cat" + "s" = cats

Derivational morphology

Messy stuff that nobody ever "taught" when learning English

- wide variety of affixes
 - -al, -tion, -ee, -er, -ant, -ness
 - anti-, un-, non-
- quasi-systematic rules
- irregular meaning change

- changes of word class

2.1.1 Language classification

- **Genetically:** groups based on family history
- **Areal:** geographical occurrences
- **Typological:** shares grammar, morphology, phonology, syntax, etc.

2.1.2 Morphological Typology

Groups languages based on aspects of morphology

Isolating vs Synthetic

- **Isolating:** one morpheme per word
- **Synthetic:** lots of morphemes per word

Agglutinating vs fusional

- **Agglutinating:** form words by concatenating stems with affixes to form words (with prefixes, suffixes, infixes)
- **Fusional:** allow morphemes to change form upon inflection; harder to isolate them

Computational morphology

Morphological analysis takes surface form and returns the stem with a set of morphological features rather than segmenting the surface form.

2.2 So, what is a word?

2.2.1 Vocabulary

- a vocabulary is the set of words a system has access to
- the minimal units of processing in an input that has some semblance of meaning attached to it.

What should such a vocabulary (or lexicon) consist of?

- all the words in a language?
- all the words in some comprehensive dictionary?
- some subset?

2.2.2 Corpora

Words don't just appear out of nowhere. Corpora are the representative texts that are used either generically for diverse applications or designed for a specific domain or application.

1. Collect a big bunch of text.

2. Figure out the words that appear in it.
3. That's your vocabulary

Dimensions

- **Language:** dialects, creoles, pidgins, etc.
- **Provenance:** how was it obtained? Consent? Legal?
- **Genre:** Newswire, fiction, non-fiction, scientific articles, Wikipedia
- **Demographics:** Writer's age, gender, race, socioeconomic status, etc.
- **Medium:** Spoken, written, captioned, video
- **Writing system:** Over 350 systems in use

How many words are there?

N = number of mentions V = vocabulary = set types ($|V|$ is the size of the vocab)

2.3 Heap's Law

$$|V| = kN^\beta$$

Where k and β are free variables. Usually k between 10 and 100 and β between 0.6 and 0.7.

- Size of the vocabulary grows somewhat faster than the square root of the corpus size
- The rate of the vocabulary growth slows as corpus grows, but never flattens out.

2.4 Text Normalization and Tokenization

This is essentially the question of "how do we process a corpora and take it from messy to clean and something that we can analyze?"

Normalizing text can look like

- Segmenting running text into sentences
- Tokenizing sentences into words
- Normalizing word formats
- Recognizing different scripts/languages

2.4.1 Issues in Normalization

- Hong Kong $\rightarrow$ one word? two?
- m.p.g $\rightarrow$ is this even a word?
- IRA, I.R.A $\rightarrow$ expand? keep out?

2.4.2 Sentence Segmentation

Tokenization, morphology, and further syntactic processing first requires sentence segmentation

- Easy in English. Punctuation is used to mark sentence boundaries.
 - "!", "?" are relatively unambiguous.
 - Periods can be. Like "Dr.", "I.R.A", etc.

2.4.3 Multilingual issues in Normalization

English is not the only language out there, and there are several problems that need to be faced.

- Code switching – "Chinglish", "Spanglish", etc.
- Writing systems – Latin, Chinese, Arabic
- Complex morphology – words built with two or more morphemes
- Sentence segmentation – Chinese, Thai, Japanese do not use spaces to separate words

2.4.4 Tokenization

Tokenization is the process of segmenting text into tokens

Given a training corpus, subword tokenization identify the base word and subwords in a vocabulary empirically.

Tokens are essentially a (crude) approximation to morphemes. They are minimal units to which we can associate meaning in a model. In worst case scenarios, a word can be decomposed into characters.

For tokenization to work you need two things:

- A learner that takes the raw training corpus and makes a vocabulary of a specific size
- A tokenizer (encoder) that takes input text and tokenizes it according to the vocabulary

2.5 Byte-Pair Encoding (BPE)

Byte Pair Encoding works by merging neighboring tokens k times to create longer and longer tokens.

The core of the algorithm is

- Choose the most frequently occurring adjacent pair of vocabulary items in the training corpus
- Add the newly merged symbol to the vocabulary
- Replace every occurrence of the adjacent pair with said merged symbol
- Redo the counts
- Repeat k times

BPE Example

Let's see how BPE really works with this sentence that will never exist beyond this example in the English language (see: infinite monkeys with infinite typewriters)

set_new_new_renew_reset_renew

First, we can break up the corpus into words, with their counts (including whitespace!).

corpus	vocabulary
2 _ n e w	_, e, n, r, s, t, w
2 _ r e n e w	
1 s e t	
1 _ r e s e t	

The BPE will iterate and counts all pairs of adjacent symbols. The most frequent in this corpus is **ne** which occurs in "new" and "renew". We merge those symbols and recount

corpus	vocabulary
2 _ne w	_, e, n, r, s, t, w, ne
2 _r e ne w	
1 s e t	
1 _ r e s e t	

Now the most frequent pair is **ne + w**. So we merge and recount

corpus	vocabulary
2 _new	_, e, n, r, s, t, w, ne, new
2 _r e new	
1 s e t	
1 _ r e s e t	

Merge 3 & 4 are **_r** merge and **_re** merge respectively.

corpus	vocabulary
2 _new	_, e, n, r, s, t, w, ne, new, _r, _re
2 _re new	
1 s e t	
1 _re s e t	

If we do that enough times merges 5-8 look like this

merge	vocabulary
(_, new)	_, e, n, r, s, t, w, ne, new, _r, _re, _new
(_re, new)	_, e, n, r, s, t, w, ne, new, _r, _re, _new, _renew
(s, e)	_, e, n, r, s, t, w, ne, new, _r, _re, _new, _renew, se
(se, t)	_, e, n, r, s, t, w, ne, new, _r, _re, _new, _renew, se, set

2.5.1 Multilingual issues with Tokenization

Most models are assumed to be monolingual. Multilingual models were built for the use of translation, retrieval, etc. While most frontier models have "multilingual" capabilities, they weren't built that way. It just so happens that when you treat the Internet akin to a kid in a candy store, the models "learn" other languages just by happenstance.

2.5.2 Does vocab size matter?

It does! Smaller vocabularies have lower memory requirements, but they might miss more tokens. But smaller vocabularies lead to longer inputs, because the smaller words lead to more tokens (commercial LLMs charge by token).

3 N-Gram Language Models

3.1 What is a language model?

A language model is a machine learning model that predicts words. More elegantly, it assigns probability to each possible next word, or gives a probability distribution of upcoming words that could may or may not be possible.

3.1.1 Word Prediction

Given a certain word, we can formalize this prediction of "given this word, what is the probability of this word occurring next?" as

$$P(w_n|w_{1:n-1})$$

With the definition of conditional probabilities

$$P(B|A) = \frac{P(A, B)}{P(A)}$$
$$P(A, B) = P(A)P(B|A)$$

...as well as the **Chain Rule** we can think of word prediction as a sequence of a joint event

$$P(w_{1:n}) = P(w_1)P(w_2|w_1)P(w_3|w_2, w_1) \dots P(w_n|w_1 \dots w_{1:n-1})$$
$$= \prod_{i=1}^n P(w_i|w_{1:i-1})$$

Problem: The Chain Rule can suggest that the probability of a given word consists of the joint probability of multiplying a number of conditional probabilities, but language is creative, and a sentence can really just appear out of nowhere. (There are definitely some sentences that I have had the horror of experiencing that I don't ever want to experience again).

3.1.2 The Markov assumption

Ok, so we can't count the probability of *all* the previous words, but what about about narrowing that scope? What about if we predict a word based on the last word? Last two? Three?

The bigram model

The bigram model gives us a probability of the next word given **just** the previous word $P(w_n|w_{n-1})$.

The assumption that the probability of a word depends just on the previous word is called a **Markov** assumption.

General cases

Bigram version

$$P(w_i|w_{i:1}) \approx P(w_i|w_{i:i-1})$$

Trigram version

$$P(w_i|w_{i:1}) \approx P(w_i|w_{i-2:i-1})$$

General case

$$P(w_i|w_{i:1}) \approx P(w_i|w_{i-(N-1):i-1})$$

Combining this with the Markov assumption and we get

$$P(w_{1:n}) \approx \prod_{i=1}^n P(w_i|w_{i-(N-1):i-1})$$

3.2 Smoothing

Language is such an interesting thing. Because of this, when training a language model, sometimes a probability of two words being together like "sincere politician" may appear in the test set, but not the training set! This would make the probability that bigram 0.

To deal with this we can use smoothing which is just – as Professor Martin put it – stealing too much from the rich and giving it to the poor.

3.2.1 Additive smoothing

Add a little bit to all the counts and the zeroes magically go away!

For unigrams it would look like

$$P_{+1}w = \frac{\text{Count}(w) + 1}{M + V}$$

For bigrams

$$P_{+1}(w_n|w_{n-1}) = \frac{\text{Count}(w_{n-1}, w_n) + 1}{\text{Count}(w_{n-1}) + V}$$

3.3 Evaluating Language Models

Best way to evaluate a model is to embed it into an application and measure its performance. This is called **extrinsic evaluation**. It's really the only way to know how well a language model will do with a given task. However, this is very expensive. So to cut costs we can do **intrinsic evaluation** which is testing the language model outside the application.

To do this, we have to have three datasets: **training set**, **dev set**, **test set**.

The **training set** is the dataset we use to train the model on. Like a study guide for an exam.

The **test set** is the exam itself. It's what is used to evaluate the model. This dataset is distinct from the training set because we don't want the model to memorize the exam answers. We want it to study, and then test its knowledge with data it's never seen before.

The **development set** is the interlude test set which we can evaluate the performance of our model before we actually use the test set. It's like a "mock-exam" per se.

3.4 Log probabilities

Using raw probabilities are related to the length of sequences and number of sentences in the test set. The more probabilities we multiply together, the smaller the number becomes.

To deal with this, we use the log space to normalize and get results that are not as small.

$$H(T) = -\frac{1}{N} \sum_{i=1}^N \log P(w_i|w_{1:i-1})$$

...where $H(T)$ is the entropy.

3.4.1 Perplexity and entropy

Perplexity is traditionally THE intrinsic measure used to evaluate large language models (like the AI model Perplexity!), especially with machine translation and speech recognition.

The perplexity of a model is 2 raised to the entropy!

$$PP(T) = 2^{H(T)}$$

Perplexity is roughly the average number of choices that a model would have to make in guessing the next item in a sequence.

- if it is 100% certain then the perplexity is 1 (entropy of 0 since $\log(1)=0$), because there is no choice.
- if it has no information and there are M choices, the perplexity is M which makes sense! it's just guessing randomly.

4 Logistic Regression and Text Classification